

Practice Sheet 2

SECTION 02

Warm-Up Questions

Concept recall questions

1. What is the difference between a list and a tuple, beyond "one is mutable and one isn't"? Why would you ever choose a tuple?

Answer- List: It is slower than a tuple. There is no hashing in a list. Tuple: it is faster than a list. there is hashing in a tuple. tuple would be chosen as it is hashable and also it is faster.

2. Why is dictionary lookup $O(1)$ on average? What data structure makes this possible internally?

Answer- Lookup is $O(1)$ on average because internally python maintains a hashtable, the key's hash value determines an index in an array. So, it can be accessed in a constant time. it uses hashtable that makes it possible internally

3. What does "hashable" mean, and which built-in types are NOT hashable?

Answer- It means an object has a stable hash value and equality comparison. int, float, str, tuple can be hashable. list, dict, set cannot be hashable.

4. What is the difference between dict.keys() / values() / items() and a list — are they "live" or a snapshot?

Answer- dict methods (keys(), values(), and items()) return live, dynamic view objects rather than static snapshot lists. lists are static snapshots.

5. Why is set membership testing faster than list membership testing?

Answer- This is because list is implemented as a dynamic array whereas set is implemented as a hash table. So, list scan each element one by one until it finds a match whereas set computes the hash of the element and jumps directly to the slot where it should be stored.

6. What happens in memory when you write `b = a` where `a` is a list? Draw or describe it.

Answer- when `b = a` is written, no new list is created. Instead, both `a` and `b` become references to the same list object in memory. Python stores the list in a block of memory. The variable name `a` points to that block. So, When `b = a` is assigned, Python just makes `b` point to the same block it doesn't copy the contents.

7. What is the difference between a list comprehension and a generator expression — both syntactically and behaviourally?

Answer-The syntax of list comprehension is `[xx for x in range(5)]` whereas the syntax for generator expression is `(xx for x in range(5))`. list comprehension builds the entire list in memory immediately whereas generator expression produces values lazily, one at a time, when iterated.

8. Name three things collections.Counter can do that you would otherwise write a manual loop for.

Answer- The three things collections.Counter can do: Frequency counting: Automatically tally occurrences of elements. Most common elements: Quickly get the top N frequent items. Arithmetic on counters: Add, subtract, or combine counts across datasets.

Rapid fire questions

1. Which data structure would you use to remove duplicates from a list while preserving order?

Answer- dict

2. Which data structure has guaranteed $O(1)$ average lookup by key?

Answer- dictionary

3. What error do you get if you try to use a list as a dict key?

Answer-TypeError: unhashable type: 'list'

4. What is the output of `len({1, 1, 2, 2, 3})` ?

Answer-3

5. What method safely retrieves a dict value without raising KeyError ?

Answer- .get()

6. What is the time complexity of `x` in `my_set` for a set of size `n`?

Answer- $O(1)$ average

7. Which comprehension type uses curly braces AND a colon?

Answer-Dict comprehension

8. What is the namedtuple field access syntax — `.field` or `["field"]` ?

Answer- .field

Fill in the blanks

- Complete to safely get a value with a default of 0 `count = my_dict.get("apple", 0)`
- Complete to merge two dicts (Python 3.9+ syntax) `merged = dict_a | dict_b`
- Complete to create an independent copy of a list of lists `import copy deep_copy = copy.deepcopy(nested_list)`
- Complete to count word frequencies in one line from collections `import Counter word_counts = Counter(words)`
- Complete to create a dict where missing keys auto-initialize to an empty list from collections `import Counter groups = Counter(list)`

SECTION 03

Predict the Output

Easy

Predict the output: (1, 2, 3, 1, 2, 3)

```
(1, 2, 3, 4)
```

```
t = (1, 2, 3)
```

```
print(t * 2)
```

```
print(t + (4,))
```

Predict the output: ['a', 'b']

```
{'a': 1, 'b': 2, 'c': 3}
```

```
d = {"a": 1, "b": 2}
```

```
print(list(d.keys()))
```

```
d["c"] = 3
```

```
print(d)
```

Predict the output: 3

```
{1, 2, 3}
```

```
s = {1, 2, 3, 2, 1}
```

```
print(len(s))
```

```
print(s)
```

Intermediate

Predict the output: [1, 2, 3, 4]

```
True
```

```
lst = [1, 2, 3]
```

```
lst2 = lst
```

```
lst2.append(4)
```

```
print(lst)
```

```
print(lst is lst2)
```

Predict the output: {'x': 2, 'y': 3}

```
d1 = {"x": 1}
```

```
d2 = {"x": 2, "y": 3}
```

```
merged = {**d1, **d2}
```

```
print(merged)
```

Predict the output: [('apple', 3), ('banana', 2)]

```
from collections import Counter
```

```
words = ["apple", "banana", "apple", "cherry", "banana", "apple"]
```

```
print(Counter(words).most_common(2))
```

Tricky

Predict the output: [[1, 0, 0], [1, 0, 0], [1, 0, 0]]

```
matrix = [[0] * 3] * 3
```

```
matrix[0][0] = 1
```

```
print(matrix)
```

Predict the output: [1]

[1, 2]

[1, 2, 3]

```
def add_item(item, lst=[]):
```

```
    lst.append(item)
```

```
    return lst
```

```
print(add_item(1))
```

```
print(add_item(2))
```

```
print(add_item(3))
```

Predict the output: {0: 1, 1: 2, 2: 3}

```
d = {}
```

```
for i in range(3):
```

```
    d[i] = d.get(i - 1, 0) + 1
```

```
print(d)
```

● Hidden behavior

Predict the output — and explain WHY: [1, 3, 5] because removing during iteration skips elements due to index shifting.

```
my_list = [1, 2, 3, 4, 5]
```

```
for item in my_list:
```

```
    if item % 2 == 0:
```

```
        my_list.remove(item)
```

```
print(my_list)
```

Predict the output — and explain WHY: (1, 2, [3, 4, 5])

```
a = (1, 2, [3, 4])
```

```
a[2].append(5)
```

```
print(a)
```

Is 'a' still a tuple? Did it just get mutated?

--a is still a tuple, but its mutable element was mutated, so the tuple reference didn't change — only the list contents did.

Predict the output: {'a': 3, 'b': 2, 'c': 4}

```
keys = ["a", "b", "a", "c"]
```

```
values = [1, 2, 3, 4]
```

```
result = dict(zip(keys, values))
```

```
print(result)
```

What happens to duplicate keys?

--When duplicate keys appear in zip(keys, values), the last assignment wins — so "a": 1 is overwritten by "a": 3.

SECTION 04

Debugging Challenge Zone

Bug 1 — Aliasing Trap

This function is supposed to add a discount tag to a list of orders WITHOUT modifying the caller's original list. It has a bug — find it.

```
def apply_discount_tag(orders):
    tagged = orders
    tagged.append("DISCOUNT_APPLIED")
    return tagged
original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)
print("Original:", original_orders) # Did this change unexpectedly?
print("New:", new_orders)
```

Fixed it below:

```
def apply_discount_tag(orders):
    tagged = orders.copy() # independent shallow copy
    tagged.append("DISCOUNT_APPLIED")
    return tagged
original_orders = ["order_1", "order_2"]
new_orders = apply_discount_tag(original_orders)
print("Original:", original_orders)

print("New:", new_orders)
```

Bug 2 — Mutation During Iteration

This is meant to remove all 'inactive' users from a list. It silently # skips some entries. Find out why and fix it.

```
users = [
    {"name": "Asha", "status": "active"},
    {"name": "Ravi", "status": "inactive"},
    {"name": "Meena", "status": "inactive"},
    {"name": "Karan", "status": "active"},
]
for user in users:
    if user["status"] == "inactive":
        users.remove(user)
print(users) # Expect 2 active users — count what you actually get
```

Fix it below:

```
users = [u for u in users if u["status"] != "inactive"]
print(users) # [{'name': 'Asha', 'status': 'active'}, {'name': 'Karan', 'status': 'active'}]
```

Bug 3 — Shallow Copy Trap

A function is supposed to create independent backups of student records so the original roster is untouched. It doesn't work as expected.

```
roster = [{"Asha", [90, 85]}, {"Ravi", [70, 75]}]
backup = roster.copy()
backup[0][1].append(100) # adding a bonus mark to the backup only
print("Roster:", roster) # Did the original get affected too?
print("Backup:", backup)
```

Fix it below (hint: which copy function copies ALL levels?):

```
import copy
roster = [{"Asha", [90, 85]}, {"Ravi", [70, 75]}]
backup = copy.deepcopy(roster) # copies all levels
backup[0][1].append(100)
print("Roster:", roster) # [{"Asha", [90, 85]}, {"Ravi", [70, 75]}]
print("Backup:", backup) # [{"Asha", [90, 85, 100]}, {"Ravi", [70, 75]}]
```

Bug 4 — KeyError Crash

This inventory lookup crashes whenever a product ID isn't found.

Make it crash-proof without changing the overall logic.

```
inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}
def get_stock(sku):
    return inventory[sku]
print(get_stock("SKU101"))
print(get_stock("SKU999")) # <-- crashes here
```

Fix it below:

```
inventory = {"SKU101": 50, "SKU102": 12, "SKU103": 0}
def get_stock(sku):
    return inventory.get(sku, "Not Found") # safe fallback
print(get_stock("SKU101")) # 50
print(get_stock("SKU999")) # Not Found
```

Bug 5 — The N+1 Lookup Problem

This function matches orders to customer names. It works, but for 10,000 orders against 10,000 customers it becomes painfully slow. Identify WHY, and rewrite it to be fast at any scale.

```
orders = [{"order_id": 1, "customer_id": 101}, {"order_id": 2, "customer_id": 102}]
customers = [{"id": 101, "name": "Asha"}, {"id": 102, "name": "Ravi"}]
def attach_customer_name(orders, customers):
    for order in orders:
        for customer in customers: # <-- the problem is here
            if customer["id"] == order["customer_id"]:
```

```

order["customer_name"] = customer["name"]
return orders
print(attach_customer_name(orders, customers))

# Rewrite using a dict lookup below (should be O(n + m), not O(n * m)):
orders = [{"order_id": 1, "customer_id": 101}, {"order_id": 2, "customer_id": 102}]
customers = [{"id": 101, "name": "Asha"}, {"id": 102, "name": "Ravi"}]
def attach_customer_name(orders, customers):
    # Build a dictionary for O(1) lookups
    customer_map = {c["id"]: c["name"] for c in customers}
    for order in orders:
        order["customer_name"] = customer_map.get(order["customer_id"], "Unknown")
    return orders
print(attach_customer_name(orders, customers))

```

🐛 Bug 6 — Unhashable Type

This is meant to group transactions by (date, category) and accumulate
totals into a dict keyed by that pair. It throws a TypeError. Why, and fix it?

```

transactions = [
    {"date": "2026-01-01", "category": ["food"], "amount": 200},
    {"date": "2026-01-01", "category": ["food"], "amount": 150},
]
totals = {}
for t in transactions:
    key = (t["date"], t["category"]) # <-- problem
    totals[key] = totals.get(key, 0) + t["amount"]
print(totals)

```

Fix it below:

```

transactions = [
    {"date": "2026-01-01", "category": ["food"], "amount": 200},
    {"date": "2026-01-01", "category": ["food"], "amount": 150},
]
totals = {}
for t in transactions:
    key = (t["date"], tuple(t["category"])) # make category immutable
    totals[key] = totals.get(key, 0) + t["amount"]
print(totals)

```

SECTION 05

Edge Cases & Hidden Scenarios

Empty Inputs

What happens with empty containers? Predict, then run.

```
print(max([], default="no data"))
print(sum([]))
print({}.get("missing", "default"))
print(list({}.items()))
```

Solution => Predictions:

```
print(max([], default="no data")) #no data → max() uses the default when list is empty
print(sum([])) # 0 → sum of empty list is defined as 0
print({}.get("missing", "default")) # default → missing key returns the fallback
print(list({}.items())) # [] → empty dict → empty list of items
```

Duplicate Keys / Values

Dict literal with duplicate keys — what survives?

```
d = {"a": 1, "b": 2, "a": 3}
print(d)
```

Solution=> d = {"a": 1, "b": 2, "a": 3}

- Python dictionaries cannot keep duplicate keys.
- When the same key appears again, the latest assignment overwrites the previous one.
- So "a": 1 is replaced by "a": 3.
- So the dictionary becomes:

```
{'a': 3, 'b': 2}
```

 #Counter with all-identical items -->

```
from collections import Counter
print(Counter(["x", "x", "x"]))
```

 Solution=> Counter(["x", "x", "x"])
- Counter counts frequency of each element.
 "x" appears 3 times → count is 3.
 So the result is:

```
Counter({'x': 3})
```

Deeply Nested Structures

```
In [4]: ## Accessing a deeply nested key that may not exist at any level
# data = {"user": {"profile": {"address": {"city": "Pune"}}}}
## This crashes if ANY level is missing. Write a safe accessor function
## that returns None instead of raising, for any depth of missing key.
# def safe_get(d, *keys):
#     pass # implement this
# print(safe_get(data, "user", "profile", "address", "city"))
# print(safe_get(data, "user", "profile", "phone")) # should return None, not cr
```



```
# Solution=>
def safe_get(d, *keys):
    for key in keys:
        if isinstance(d, dict) and key in d:
            d = d[key]
        else:
            return None
    return d
data = {"user": {"profile": {"address": {"city": "Pune"}}}}
print(safe_get(data, "user", "profile", "address", "city"))
print(safe_get(data, "user", "profile", "phone"))
```

Pune

None

Large-Scale Behavior

```
In [ ]: import time
# Compare: membership testing in a list of 200,000 items vs a set
big_list = list(range(200_000))
big_set = set(big_list)
start = time.time()
print(199_999 in big_list)
print("list lookup:", time.time() - start, "seconds")
start = time.time()
print(199_999 in big_set)
print("set lookup:", time.time() - start, "seconds")
# Run this and observe the gap – this IS the reason sets exist.

List lookup:
- Python must scan through elements one by one until it finds a match (linear search)
- Time complexity: O(n).
- With 200,000 items, that means up to 200,000 comparisons in the worst case.

Set lookup:
- Python uses a hash table internally.
- It computes the hash of the element and jumps directly to the slot where it should be.
- Time complexity: O(1) on average.
- Even with millions of elements, lookup remains fast.
```

Iterator Exhaustion

```
In [10]: # Generators can only be consumed ONCE. Predict the second print:
gen = (x * x for x in range(5))
print(list(gen))
print(list(gen)) # what do you get the second time? why?
# First print(list(gen)) :
# The generator produces values one by one: 0, 1, 4, 9, 16.
# Second print(list(gen)) :
# - Generators are iterators that can only be consumed once.
# - After the first list(gen), the generator is exhausted – it has no more values.
# - Output: [] (an empty list).
```

[0, 1, 4, 9, 16]

[]

SECTION 06

Industry-Based Scenarios

```
In [11]: # Each scenario mirrors a real production task. Solve using the data structure(s)
# Banking
# # A bank needs to flag accounts with more than 3 transactions over ₹50,000
# # in a single day, for fraud review. Use a dict to group by account_id.
# transactions = [
#     {"account_id": "ACC1", "amount": 60000},
#     {"account_id": "ACC1", "amount": 70000},
#     {"account_id": "ACC2", "amount": 20000},
#     {"account_id": "ACC1", "amount": 55000},
#     {"account_id": "ACC1", "amount": 51000},
# ]
# # Your code: build a dict of account_id -> count of transactions > 50000,
# # then list accounts where that count > 3
# Solution =>
transactions = [
    {"account_id": "ACC1", "amount": 60000},
    {"account_id": "ACC1", "amount": 70000},
    {"account_id": "ACC2", "amount": 20000},
    {"account_id": "ACC1", "amount": 55000},
    {"account_id": "ACC1", "amount": 51000},
]
# Build dict: account_id -> count of transactions > 50,000
counts = {}
for tx in transactions:
    if tx["amount"] > 50000:
        acc = tx["account_id"]
        counts[acc] = counts.get(acc, 0) + 1
# Flag accounts with count > 3
flagged_accounts = [acc for acc, cnt in counts.items() if cnt > 3]
print("Counts:", counts)
print("Flagged accounts:", flagged_accounts)
```

Counts: {'ACC1': 4}

Flagged accounts: ['ACC1']

Healthcare

```
In [13]: # A clinic system stores patient allergies as sets so duplicate entries
# never occur. Two records for the same patient need to be merged safely.
# record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}}
# record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}

# Your code: merge the allergy sets into one combined, de-duplicated set
# Solution =>
record_a = {"patient": "P1001", "allergies": {"penicillin", "peanuts"}}
record_b = {"patient": "P1001", "allergies": {"peanuts", "latex"}}

# Merge allergy sets (union ensures no duplicates)
merged_allergies = record_a["allergies"] | record_b["allergies"]

merged_record = {
    "patient": record_a["patient"],
    "allergies": merged_allergies
}
print(merged_record)
```

```
{'patient': 'P1001', 'allergies': {'penicillin', 'latex', 'peanuts'}}
```

SECTION 07

Think Like an Engineer

Failure Analysis

For each situation, explain what will go wrong and why — before fixing anything.

1. A teammate writes `cache = {}` as a default function argument: `def fetch(key, cache={}):`. Six months later, production has a memory leak. Why?

Solution => In Python, default arguments are evaluated once at function definition time, not each call that means all calls to `fetch()` share the same dictionary object.

- This is because mutating that dict (adding keys) persists across calls, causing unbounded growth → memory leak.

- Always use `None` as the default and create a new dict inside the function.

2. A reporting script does `for k in my_dict: del my_dict[k]` to clear a dict during iteration. What happens?

Solution => You're modifying the dictionary while iterating over it. This is because Python raises a `RuntimeError: dictionary changed size during iteration`.

- Use `my_dict.clear()` or iterate over a copy of the keys (`for k in list(my_dict): del my_dict[k]`).

3. A junior dev uses a list to track "seen" user IDs across 5 million log lines, checking if `uid in seen_list`. The job that used to take 2 minutes now takes 6 hours. Diagnose it.

Solution => Checking `uid in seen_list` is $O(n)$ for lists. This is because each lookup scans the list linearly. With millions of checks, runtime explodes from minutes to hours.

- Use a set for membership testing → average $O(1)$ lookup, drastically faster.

4. Someone passes a dict as a default argument to a function that's supposed to return a fresh config each time it's called. What's the risk?

Solution => This is because instead of returning a fresh config, all callers mutate the same dict.

- Risk:
Configs bleed into each other, causing subtle bugs and incorrect behavior.

SECTION 08

Hands-On Coding Practice

Beginner

1. Write a function that takes a list of numbers and returns a NEW list

```
In [15]: # containing only the even numbers, using a list comprehension.
# def get_evens(numbers):
#     pass
# print(get_evens([1, 2, 3, 4, 5, 6]))
# Solution :
def get_evens(numbers):
    """
    Return a new list containing only the even numbers.
    """
    return [n for n in numbers if n % 2 == 0]
print(get_evens([1, 2, 3, 4, 5, 6]))
```

[2, 4, 6]

2. Write a function that takes a list of words and returns a dict

```
In [16]: # mapping each word to its length.
# def word_lengths(words):
#     pass
# print(word_lengths(["python", "ai", "ml"]))
# Solution:
def word_lengths(words):
    """
    Return a dict mapping each word to its length.
    """
    return {word: len(word) for word in words}

print(word_lengths(["python", "ai", "ml"]))
```

{'python': 6, 'ai': 2, 'ml': 2}

3. Write a function that returns True if a tuple contains any duplicate values.

```
In [17]: # def has_duplicates(t):
#     pass
# print(has_duplicates((1, 2, 3))) # False
# print(has_duplicates((1, 2, 2))) # True
# Solution:
def has_duplicates(t):
    """
    Return True if the tuple contains any duplicate values.
    """
    return len(t) != len(set(t))

# Tests
print(has_duplicates((1, 2, 3))) # False
print(has_duplicates((1, 2, 2))) # True
```

False

True

Intermediate

4. Given two lists of equal length representing student names and scores,

```
In [18]: # build a dict of name -> score, then return the name with the highest score.
# def top_scorer(names, scores):
#     pass
# print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))
# Solution:
def top_scorer(names, scores):
    """
    Build a dict of name -> score, then return the name with the highest score.
    """
    # Build dictionary mapping
    score_map = dict(zip(names, scores))

    # Find the name with the highest score
    return max(score_map, key=score_map.get)
print(top_scorer(["Asha", "Ravi", "Meena"], [88, 95, 91]))
```

Ravi

5. Write a function that flattens a list of lists (one level deep)

```
In [19]: # using a nested list comprehension.
# def flatten(list_of_lists):
#     pass
# print(flatten([[1, 2], [3, 4], [5]])) # [1, 2, 3, 4, 5]
# Solution :
def flatten(list_of_lists):
    """
    Flatten a list of lists (one level deep) using a nested list comprehension.
    """
    return [item for sublist in list_of_lists for item in sublist]
print(flatten([[1, 2], [3, 4], [5]])) # [1, 2, 3, 4, 5]
```

[1, 2, 3, 4, 5]

6. Using collections.defaultdict, group a list of words by their first letter.

```
In [20]: # from collections import defaultdict
# def group_by_first_letter(words):
#     pass
# print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))
# {'a': ['apple', 'ant'], 'b': ['banana'], 'c': ['cat', 'car']}
# Solution:
from collections import defaultdict
def group_by_first_letter(words):
    """
    Group words by their first letter using defaultdict.
    """
    grouped = defaultdict(list)
    for word in words:
        if word: # avoid empty strings
            grouped[word[0]].append(word)
    return dict(grouped) # convert back to regular dict for clean output
print(group_by_first_letter(["apple", "ant", "banana", "cat", "car"]))
```

{'a': ['apple', 'ant'], 'b': ['banana'], 'c': ['cat', 'car']}

Advanced

7. Write a function that finds all pairs of numbers in a list that sum

```
In [22]: # to a given target, using a set for O(n) time (not nested loops).
# def find_pairs_with_sum(numbers, target):
#     pass
# print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9)) # should find (2,7) and (
# Solution =
def find_pairs_with_sum(numbers, target):
    """
    Find all pairs of numbers in the list that sum to target.
    Uses a set for O(n) time complexity.
    """
    seen = set()
    pairs = []
    for num in numbers:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
            seen.add(num)
    return pairs

print(find_pairs_with_sum([2, 7, 11, 15, -2, 9], 9))
```

[(2, 7), (11, -2)]

8. Write a function that takes a list of (timestamp, event) tuples and returns a dict mapping each event type to a deque of its timestamps, keeping only the last 3 occurrences per event (use collections.deque maxlen).

```
In [23]: # from collections import defaultdict, deque
# def recent_events_by_type(events, keep=3):
#     pass
# events = [(1, "Login"), (2, "click"), (3, "Login"), (4, "Login"), (5, "click")]
# print(recent_events_by_type(events))
# # 9. Write a generator function (not a list) that lazily yields running
# # totals of a list of numbers – without ever storing the full result list.
# def running_totals(numbers):
#     pass
# print(list(running_totals([1, 2, 3, 4]))) # [1, 3, 6, 10]
# Solution:
from collections import defaultdict, deque
def recent_events_by_type(events, keep=3):
    """
    Return a dict mapping each event type to a deque of its last `keep` timestamps
    """
    grouped = defaultdict(lambda: deque(maxlen=keep))
    for ts, ev in events:
        grouped[ev].append(ts)
    return dict(grouped) # convert for clean output

events = [(1, "login"), (2, "click"), (3, "login"), (4, "login"), (5, "click"),
print(recent_events_by_type(events))
```

{'login': deque([3, 4, 6], maxlen=3), 'click': deque([2, 5], maxlen=3)}

SECTION 09

Scenario-Based Coding Tasks

Task 1 — Inventory Reconciliation

```
In [25]: # Two warehouse systems report stock counts for the same SKUs. Find SKUs
# that DISAGREE between the two systems, and by how much.
# warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20}
# warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}
# Your code:
# 1. Find SKUs present in only one warehouse
# 2. Find SKUs present in both but with different counts, and the difference
# Solution =
warehouse_a = {"SKU1": 100, "SKU2": 50, "SKU3": 20}
warehouse_b = {"SKU1": 100, "SKU2": 45, "SKU4": 10}

# 1. SKUs present in only one warehouse
only_in_a = set(warehouse_a.keys()) - set(warehouse_b.keys())
only_in_b = set(warehouse_b.keys()) - set(warehouse_a.keys())

# 2. SKUs present in both but with different counts
disagreements = {}
for sku in warehouse_a.keys() & warehouse_b.keys():
    if warehouse_a[sku] != warehouse_b[sku]:
        disagreements[sku] = abs(warehouse_a[sku] - warehouse_b[sku])

print("Only in A:", only_in_a)
print("Only in B:", only_in_b)
print("Disagreements:", disagreements)
```

```
Only in A: {'SKU3'}
Only in B: {'SKU4'}
Disagreements: {'SKU2': 5}
```

SECTION 10

Production Thinking Challenges

Scale

A function processes a list of 50 million transaction records and does if txn_id in processed_list inside the loop, where processed_list grows with each iteration. Identify the complexity class of this approach overall, and rewrite the approach description (no need to fully code it) using the right structure. Solution= Membership check txn_id in processed_list is O(n) because it scans linearly.

- As processed_list grows, each lookup gets slower.
- Overall complexity: O(n²) for 50 million records → catastrophic performance (trillions of comparisons).
- Rewritten Description: Instead of growing a list and checking membership linearly, maintain a set of processed transaction IDs. For each transaction, check membership in the set (constant time) and insert into the set if new. This reduces the

reconciliation from quadratic time to linear time, making it feasible for 50 million records in production.

Reliability

```
In [ ]: # A configuration loader does settings = config_dict["env"]["region"]["timeout"]
# occasionally crashes with KeyError because a partial config was deployed. Desc
# (without using try/except) that would make this robust, and the tradeoff of ea
# Solution =
# Defensive Strategy 1 – Each .get() returns a dict or a default. If a key is mi
# Use .get() chaining
settings = (
    config_dict.get("env", {})
        .get("region", {})
        .get("timeout", DEFAULT_TIMEOUT)
)

# Defensive Strategy 2 – Validate upfront
required_keys = ["env", "region", "timeout"]
current = config_dict
for key in required_keys:
    if key not in current:
        raise ValueError(f"Missing required config key: {key}")
    current = current[key]
settings = current
How it works: Walks through the required keys step by step, explicitly checking
```

SECTION 11

Code Improvement Exercises

Refactor 1

```
In [32]: # Refactor this to use a dict comprehension instead of a loop
# result = {}
# for word in ["python", "java", "go"]:
#     result[word] = len(word)
# Your refactor:
result = {word: len(word) for word in ["python", "java", "go"]}
```

Refactor 2

```
In [33]: # Refactor this O(n^2) duplicate-finder to run in O(n) using a set
# def has_duplicate(nums):
#     for i in range(len(nums)):
#         for j in range(i + 1, len(nums)):
#             if nums[i] == nums[j]:
#                 return True
#     return False
# Your refactor:
def has_duplicate(nums):
    """
    Return True if the list contains any duplicate values.
    Uses a set for O(n) time complexity.
    """
```



```

seen = set()
for num in nums:
    if num in seen:
        return True
    seen.add(num)
return False

```

Refactor 3

```

In [ ]: # Refactor this manual counting loop to use collections.Counter
# counts = {}
# for tag in ["bug", "feature", "bug", "bug", "chore", "feature"]:
#     if tag in counts:
#         counts[tag] += 1
#     else:
#         counts[tag] = 1
# Your refactor

from collections import Counter
# Refactor 3
counts = Counter(["bug", "feature", "bug", "bug", "chore", "feature"])

```

Refactor 4 — Edge Case Handling

```

In [34]: # This works for the happy path but crashes on empty input or missing keys.
# Refactor to handle both gracefully.
# def average_score(students):
#     total = 0
#     for s in students:
#         total += s["score"]
#     return total / len(students)
# print(average_score([])) # currently crashes – fix it
# print(average_score([{"name": "Asha"}])) # currently crashes – fix it
# Solution:
def average_score(students):
    """
    Compute the average score of students.
    Returns None if no valid scores are found.
    """
    # Collect scores only if the key exists
    scores = [s["score"] for s in students if "score" in s]
    if not scores: # empty list or no valid scores
        return None
    return sum(scores) / len(scores)

# Tests
print(average_score([])) # None
print(average_score([{"name": "Asha"}])) # None
print(average_score([{"name": "Asha", "score": 90}, {"name": "Ravi", "score": 80}]))

```

```

None
None
85.0

```

SECTION 13**Viva Questions****Academic**

1. Define mutability and immutability with one example data structure each.

Answer=

- Mutability: A mutable object can be changed after creation. Example: List → `a = [1,2,3]; a[0] = 99 → [99,2,3]`.
- Immutability: An immutable object cannot be altered once created. Example: Tuple → `t = (1,2,3); t[0] = 99 → will give error`.

2. What is the difference between `is` and `==` when comparing two lists with identical contents?

Answer-

- `is` → checks object identity: whether both variables point to the exact same object in memory.
- `==` → checks value equality: whether the contents of the two lists are the same, regardless of whether they are stored in different objects.

3. Explain insertion order guarantee in Python dictionaries and since which version it became official language behavior

Answer-

- Before Python 3.6: Dictionaries preserved insertion order as an implementation detail (not guaranteed).
- Python 3.7 onwards: Insertion order became an official language guarantee.

In []: